



电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Written Assignment

Course Title: Embedded Processors UESTC & UESTCHN 2004

School of Engineering, University of Glasgow

Project Title: Patient CareGuard: Temperature Monitoring and Fall Detection System

Name:Chen YiChi

Student GUID: [redacted]

Content

1. Introduction----------3

2. Methodology----------3

3. Results and Discussions----------15

4. Conclusions and Future Work----------17

5. Reference----------17

1. Introduction

A growing population of elderly individuals is experiencing a range of physical and mental challenges. As a result, they are more prone to the risk of falling and suffer from fever [1]. The motivation of this project is to design an energy-efficient patient system that can monitor elderly's body temperature in real time and detect falls. This was accomplished through the employment of the NUCLEO L432KC mbed microcontroller development platform, in conjunction with the TMP102 digital temperature transducer and the ADXL345 accelerometer for motion detection. In the project, buzzer and OLED were also employed to give a warning when unusual incidents occur.

2. Methodology

Hardware

The hardware of the project included an NUCLEO L432KC mbed chip, a TMP102 digital temperature transducer, an ADXL345 accelerometer, a buzzer, an Oled screen, several Dupont thread and wires. The software used in the project was Mbed studio.

In the whole project, the NUCLEO L432KC mbed chip worked as a terminal and platform which could not only store and process data from TMP102 digital temperature transducer and the ADXL345 accelerometer but also give commands to buzzer and OLED.

The TMP102 digital provided real-time temperature detection to check whether elderly were suffering from fever. It utilized I2C communication to interface with the NUCLEO L432KC mbed chip.

The ADXL345 is a compact, thin, energy-efficient triaxial accelerometer

designed to detect accelerations in the range of +/-16g [2]. This device interfaces with the host system via the Serial Peripheral Interface (SPI) protocol.

The buzzer is an electrical device that produces a buzzing sound as a signal. Its different frequencies can be realized by setting square waves with different duty cycles. The buzzer was employed to sound an alarm when the patient was in unusual situation.

The OLED display is a display made using Organic Light Emitting Diodes (OLEDs), characterized by high contrast, high response speed, and low power consumption. OLEDs consume less power than other displays, making them an energy-efficient choice for sustainable applications. In the project, the display was used to demonstrate an alert when incidents happen.

The circuit was connected as shown in Figure 1.

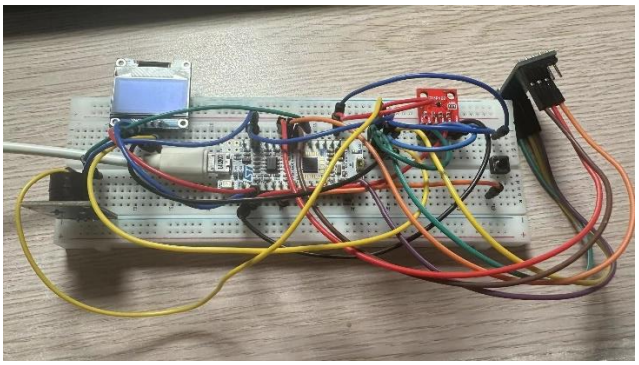
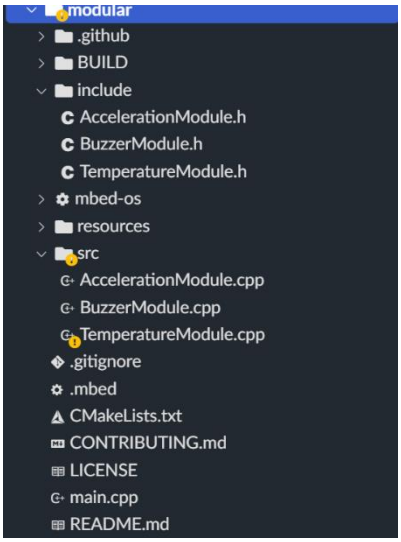


Figure 1

software

The code was modularized. It had three heading files, three source module



and a main program as shown in Figure 2. It was compiled by mbed.

Figure 2

AccelerationModule.h defined the values and function used in AccelerationModule.cpp. AccelerationModule.cpp Initialized the SPI communication pins and chip select pin for the accelerometer. When Chip select(CS) pin was low, the chip read acceleration data from the accelerometer and converted it into floating-point values for X, Y, and Z axes, scaling the raw data by 0.004 to represent acceleration in g's.

Code of AccelerationModule.h

```
#ifndef ACCELERATIONMODULE_H
#define ACCELERATIONMODULE_H

#include "mbed.h"

class AccelerationModule {
public:
    AccelerationModule(PinName mosi, PinName miso, PinName sclk, PinNam
e cs);
    void initialise();
    void readAcceleration(float& x, float& y, float& z);

private:
```

```
    SPI ADXL345;
    DigitalOut ADXL345_cs;
    char buffer[6];
    int16_t data[3];
};

#endif // ACCELERATIONMODULE_H
```

Code of AccelerationModule.CPP

```
#include "include/AccelerationModule.h"

AccelerationModule::AccelerationModule(PinName mosi, PinName miso, PinN
ame sclk, PinName cs)
    : ADXL345(mosi, miso, sclk), ADXL345_cs(cs) {}

void AccelerationModule::initialise() {
  ADXL345_cs=1;
    ADXL345.format(8,3);
    ADXL345.frequency(2000000);
    ADXL345_cs=0;
    ADXL345.write(0x31);
    ADXL345.write(0x0B);
    ADXL345_cs=1;
    ADXL345_cs=0;
    ADXL345.write(0x2D);
    ADXL345.write(0x08);
    ADXL345_cs=1;

}

void AccelerationModule::readAcceleration(float& x, float& y, float& z)
 {
        ADXL345_cs=0;
    ADXL345.write(0x80|0x40|0x32);
    for(int i=0; i<5; i++){
        buffer[i] = ADXL345.write(0x00);
    }
    ADXL345_cs = 1;
    data[0] = buffer[1]<<8 | buffer[0];
    data[1] = buffer[3]<<8 | buffer[2];
    data[2] = buffer[5]<<8 | buffer[4];
    x=0.004*data[0];
    y=0.004*data[1];
    z=0.004*data[2];
```

```
    printf("x=%f\n",x);
     printf("y=%f\n",y);
      printf("z=%f\n",z);
}
```

BuzzerModule.h defined the values and functions used in BuzzerModule.Cpp.The cpp code set the PWM period to generate the desired frequency by calculating `1 / (2 * frequency)`.It also set the duty cycle of the PWM to 0.5, which controled the volume. Beside, It used a `wait_us` function to pause for the duration specified by the beat, effectively controlling the rhythm.

Code of BuzzerModule.h

```
#ifndef BUZZERMODULE_H
#define BUZZERMODULE_H

#include "mbed.h"

class BuzzerModule {
public:
    BuzzerModule(PinName pwm);
    void alarm(const float frequency[], const float beat[], int size);

private:
    PwmOut buzzer;
};

#endif // BUZZERMODULE_H
```

Code of BuzzerModule.cpp

```
#include "include/BuzzerModule.h"
#include "mbed.h"


BuzzerModule::BuzzerModule(PinName pwm) : buzzer(pwm) {}

void BuzzerModule::alarm(const float frequency[], const float beat[], i
nt size) {
      for(int i=0;i<=27;i++){
        //Change PWM's period to change the frequency of the music
        buzzer.period(1/(2*frequency[i]));
        //duty cycle
```

```
        buzzer=0.5;
        //time interruption for beat
        wait_us(1000000*0.4*beat[i]);
    }
}
```

TemperatureModule.h defined the values and functions used in TemperatureModule.cpp. It had three parts. The first part initialized the base class TMP102 with the I2C SDA and SCL pins, and set the default I2C address for the TMP102 sensor. The second part set up the TMP102 sensor by writing to its configuration registers. It prepared an array ConfigRegisterTMP102 with the desired configuration values and sent them to the TMP102 sensor by writing to its I2C address. The third part called readTemperature read two bytes from the TMP102 sensor's temperature registers and then combined these bytes to calculate the temperature value. Lastly, it converted the raw temperature data into a floating-point number representing the temperature in degrees Celsius.

Code of TemperatureModule.h

```
#ifndef TemperatureModule_H
#define TemperatureModule_H

#include "mbed.h"

class TemperatureModule {
public:
    TemperatureModule(PinName sda, PinName scl);
    void configure();
    float readTemperature();
    I2C i2c;

private:

    char tmp102Address;
    char configRegister[3];
    char tempRegister[2];
};

#endif
```

Code of TemperatureModule.cpp

```
#include "include/TemperatureModule.h"

TemperatureModule::TemperatureModule(PinName sda, PinName scl) : i2c(sd
a, scl), tmp102Address(0x48) {}

void TemperatureModule::configure() {
    configRegister[0] = 0x01;
    configRegister[1] = 0x60;
    configRegister[2] = 0xA0;
    i2c.write(tmp102Address << 1, configRegister, 3);
}

float TemperatureModule::readTemperature() {
    char tempRegAddr = 0x00;
    i2c.write(tmp102Address << 1, &tempRegAddr, 1);
    i2c.read(tmp102Address << 1, tempRegister, 2);

    unsigned short M = tempRegister[0] << 4;
    char L = tempRegister[1] >> 4;
    float temperature = (M + L) * 0.0625;
    printf("temperature%2f\n",temperature);
    return temperature;
}
```

In the main file, It defined the commands and font of OLED display. Constants for screen dimensions and various commands to control the SSD1306 display were also defined. An `I2C` object was created for communication with I2C devices. In the main function. Instances of TemperatureModule, AccelerationModule, and BuzzerModule were created. The temperature sensor and accelerometer were initialized, and the SSD1306 display was set up. A continuous loop read temperature and acceleration data. If an anomaly was detected (temperature out of the range or high acceleration values), the loop would break and display an "Alert" on the OLED screen. After the loop, the buzzer played a tune defined by frequency1 and beat1.

Code of Main.cpp

```
#include "mbed.h"

#include "include/TemperatureModule.h"
#include "include/AccelerationModule.h"
#include "include/BuzzerModule.h"
//for the frequency
float frequency1[]={784,659,784,1046.5,880,
1046.5,784,784,532,587,659,587,532,587,
784,659,784,1046.5,988,880,1046.5,784,784,
587,659,698.5,494,532};

//for the beat
float beat1[]={1,0.5,0.5,2,1,1,2,1,0.5,0.5,1,0.5,0.5,4,
1,0.5,0.5,1.5,0.5,0.5,1,1,2,1,0.5,0.5,1.5,0.5,4};
//the music
//end buzzer
// I2C pin definitions
#define SDA D0
#define SCL D1

// SSD1306 I2C address (shifted left by 1 bit for mbed I2C library)
#define SSD1306_I2C_ADDRESS 0x3C << 1

// Display dimensions
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

// Command definitions
#define SSD1306_DISPLAY_OFF 0xAE
#define SSD1306_DISPLAY_ON 0xAF
#define SSD1306_SET_DISPLAY_CLOCK_DIV 0xD5
#define SSD1306_SET_MULTIPLEX 0xA8
#define SSD1306_SET_DISPLAY_OFFSET 0xD3
#define SSD1306_SET_START_LINE 0x40
#define SSD1306_CHARGE_PUMP 0x8D
#define SSD1306_MEMORY_MODE 0x20
#define SSD1306_SEG_REMAP 0xA1
#define SSD1306_COM_SCAN_DEC 0xC8
#define SSD1306_SET_CONTRAST 0x81
#define SSD1306_SET_PRECHARGE 0xD9
#define SSD1306_SET_VCOM_DETECT 0xDB
#define SSD1306_DISPLAY_ALL_ON_RESUME 0xA4
#define SSD1306_NORMAL_DISPLAY 0xA6
```

```
I2C i2c(SDA, SCL);

// 5x7 Font array (ASCII characters 32 to 127)
static const char font[] = {
    // Define a simple font array for demonstration
    // Space ' ' to '~' characters in 5x7 font
    0x00, 0x00, 0x00, 0x00, 0x00, // ' '
    0x00, 0x00, 0x5F, 0x00, 0x00, // '!'
    0x00, 0x07, 0x00, 0x07, 0x00, // '"'
    0x14, 0x7F, 0x14, 0x7F, 0x14, // '#'
    0x24, 0x2A, 0x7F, 0x2A, 0x12, // '$'
    0x23, 0x13, 0x08, 0x64, 0x62, // '%'
    0x36, 0x49, 0x55, 0x22, 0x50, // '&'
    0x00, 0x05, 0x03, 0x00, 0x00, // '''
    0x00, 0x1C, 0x22, 0x41, 0x00, // '('
    0x00, 0x41, 0x22, 0x1C, 0x00, // ')'
    0x14, 0x08, 0x3E, 0x08, 0x14, // '*'
    0x08, 0x08, 0x3E, 0x08, 0x08, // '+'
    0x00, 0x50, 0x30, 0x00, 0x00, // ','
    0x08, 0x08, 0x08, 0x08, 0x08, // '-'
    0x00, 0x60, 0x60, 0x00, 0x00, // '.'
    0x20, 0x10, 0x08, 0x04, 0x02, // '/'
    0x3E, 0x51, 0x49, 0x45, 0x3E, // '0'
    0x00, 0x42, 0x7F, 0x40, 0x00, // '1'
    0x42, 0x61, 0x51, 0x49, 0x46, // '2'
    0x21, 0x41, 0x45, 0x4B, 0x31, // '3'
    0x18, 0x14, 0x12, 0x7F, 0x10, // '4'
    0x27, 0x45, 0x45, 0x45, 0x39, // '5'
    0x3C, 0x4A, 0x49, 0x49, 0x30, // '6'
    0x01, 0x71, 0x09, 0x05, 0x03, // '7'
    0x36, 0x49, 0x49, 0x49, 0x36, // '8'
    0x06, 0x49, 0x49, 0x29, 0x1E, // '9'
    0x00, 0x36, 0x36, 0x00, 0x00, // ':'
    0x00, 0x56, 0x36, 0x00, 0x00, // ';'
    0x08, 0x14, 0x22, 0x41, 0x00, // '<'
    0x14, 0x14, 0x14, 0x14, 0x14, // '='
    0x00, 0x41, 0x22, 0x14, 0x08, // '>'
    0x02, 0x01, 0x51, 0x09, 0x06, // '?'
    0x32, 0x49, 0x79, 0x41, 0x3E, // '@'
    0x7E, 0x11, 0x11, 0x11, 0x7E, // 'A'
    0x7F, 0x49, 0x49, 0x49, 0x36, // 'B'
    0x3E, 0x41, 0x41, 0x41, 0x22, // 'C'
    0x7F, 0x41, 0x41, 0x22, 0x1C, // 'D'
```

```
    0x7F, 0x49, 0x49, 0x49, 0x41, // 'E'
    0x7F, 0x09, 0x09, 0x09, 0x01, // 'F'
    0x3E, 0x41, 0x49, 0x49, 0x7A, // 'G'
    0x7F, 0x08, 0x08, 0x08, 0x7F, // 'H'
    0x00, 0x41, 0x7F, 0x41, 0x00, // 'I'
    0x20, 0x40, 0x41, 0x3F, 0x01, // 'J'
    0x7F, 0x08, 0x14, 0x22, 0x41, // 'K'
    0x7F, 0x40, 0x40, 0x40, 0x40, // 'L'
    0x7F, 0x02, 0x0C, 0x02, 0x7F, // 'M'
    0x7F, 0x04, 0x08, 0x10, 0x7F, // 'N'
    0x3E, 0x41, 0x41, 0x41, 0x3E, // 'O'
    0x7F, 0x09, 0x09, 0x09, 0x06, // 'P'
    0x3E, 0x41, 0x51, 0x21, 0x5E, // 'Q'
    0x7F, 0x09, 0x19, 0x29, 0x46, // 'R'
    0x46, 0x49, 0x49, 0x49, 0x31, // 'S'
    0x01, 0x01, 0x7F, 0x01, 0x01, // 'T'
    0x3F, 0x40, 0x40, 0x40, 0x3F, // 'U'
    0x1F, 0x20, 0x40, 0x20, 0x1F, // 'V'
    0x3F, 0x40, 0x38, 0x40, 0x3F, // 'W'
    0x63, 0x14, 0x08, 0x14, 0x63, // 'X'
    0x07, 0x08, 0x70, 0x08, 0x07, // 'Y'
    0x61, 0x51, 0x49, 0x45, 0x43, // 'Z'
    0x00, 0x7F, 0x41, 0x41, 0x00, // '['
    0x02, 0x04, 0x08, 0x10, 0x20, // '\'
    0x00, 0x41, 0x41, 0x7F, 0x00, // ']'
    0x04, 0x02, 0x01, 0x02, 0x04, // '^'
    0x40, 0x40, 0x40, 0x40, 0x40, // '_'
    0x00, 0x01, 0x02, 0x04, 0x00, // '`'
    0x20, 0x54, 0x54, 0x54, 0x78, // 'a'
    0x7F, 0x48, 0x44, 0x44, 0x38, // 'b'
    0x38, 0x44, 0x44, 0x44, 0x20, // 'c'
    0x38, 0x44, 0x44, 0x48, 0x7F, // 'd'
    0x38, 0x54, 0x54, 0x54, 0x18, // 'e'
    0x08, 0x7E, 0x09, 0x01, 0x02, // 'f'
    0x0C, 0x52, 0x52, 0x52, 0x3E, // 'g'
    0x7F, 0x08, 0x04, 0x04, 0x78, // 'h'
    0x00, 0x44, 0x7D, 0x40, 0x00, // 'i'
    0x20, 0x40, 0x44, 0x3D, 0x00, // 'j'
    0x7F, 0x10, 0x28, 0x44, 0x00, // 'k'
    0x00, 0x41, 0x7F, 0x40, 0x00, // 'l'
    0x7C, 0x04, 0x18, 0x04, 0x7C, // 'm'
    0x7C, 0x08, 0x04, 0x04, 0x78, // 'n'
    0x38, 0x44, 0x44, 0x44, 0x38, // 'o'
    0x7C, 0x14, 0x14, 0x14, 0x08, // 'p'
```

```
    0x08, 0x14, 0x14, 0x18, 0x7C, // 'q'
    0x7C, 0x08, 0x04, 0x04, 0x08, // 'r'
    0x48, 0x54, 0x54, 0x54, 0x20, // 's'
    0x04, 0x3F, 0x44, 0x40, 0x20, // 't'
    0x3C, 0x40, 0x40, 0x20, 0x7C, // 'u'
    0x1C, 0x20, 0x40, 0x20, 0x1C, // 'v'
    0x3C, 0x40, 0x30, 0x40, 0x3C, // 'w'
    0x44, 0x28, 0x10, 0x28, 0x44, // 'x'
    0x0C, 0x50, 0x50, 0x50, 0x3C, // 'y'
    0x44, 0x64, 0x54, 0x4C, 0x44, // 'z'
    0x00, 0x08, 0x36, 0x41, 0x00, // '{'
    0x00, 0x00, 0x7F, 0x00, 0x00, // '|'
    0x00, 0x41, 0x36, 0x08, 0x00, // '}'
    0x08, 0x08, 0x2A, 0x1C, 0x08, // '←'
    0x08, 0x1C, 0x2A, 0x08, 0x08, // '→
};

void ssd1306_write_command(uint8_t command) {
    char data[2];
    data[0] = 0x00; // Co = 0, D/C# = 0
    data[1] = command;
    i2c.write(SSD1306_I2C_ADDRESS, data, 2);
}

void ssd1306_init() {
    ssd1306_write_command(SSD1306_DISPLAY_OFF);
    ssd1306_write_command(SSD1306_SET_DISPLAY_CLOCK_DIV);
    ssd1306_write_command(0x80);
    ssd1306_write_command(SSD1306_SET_MULTIPLEX);
    ssd1306_write_command(SCREEN_HEIGHT - 1);
    ssd1306_write_command(SSD1306_SET_DISPLAY_OFFSET);
    ssd1306_write_command(0x00);
    ssd1306_write_command(SSD1306_SET_START_LINE | 0x00);
    ssd1306_write_command(SSD1306_CHARGE_PUMP);
    ssd1306_write_command(0x14);
    ssd1306_write_command(SSD1306_MEMORY_MODE);
    ssd1306_write_command(0x00);
    ssd1306_write_command(SSD1306_SEG_REMAP | 0x01);
    ssd1306_write_command(SSD1306_COM_SCAN_DEC);
    ssd1306_write_command(SSD1306_SET_CONTRAST);
    ssd1306_write_command(0x8F);
    ssd1306_write_command(SSD1306_SET_PRECHARGE);
    ssd1306_write_command(0xF1);
    ssd1306_write_command(SSD1306_SET_VCOM_DETECT);
```

```
    ssd1306_write_command(0x40);
    ssd1306_write_command(SSD1306_DISPLAY_ALL_ON_RESUME);
    ssd1306_write_command(SSD1306_NORMAL_DISPLAY);
    ssd1306_write_command(SSD1306_DISPLAY_ON);
}

void ssd1306_clear() {
    for (uint16_t i = 0; i < (SCREEN_WIDTH * SCREEN_HEIGHT / 8); i++) {
        char data[2];
        data[0] = 0x40; // Co = 0, D/C# = 1
        data[1] = 0x00;
        i2c.write(SSD1306_I2C_ADDRESS, data, 2);
    }
}

void ssd1306_draw_char(int x, int y, char c) {
    // Adjust the font array to fit the ASCII range
    if (c < ' ' || c > 'z') {
        c = ' '; // Replace unsupported characters with a space
    }
    c -= ' ';

    for (int i = 0; i < 5; i++) {
        char data[2];
        data[0] = 0x40; // Co = 0, D/C# = 1
        data[1] = font[c * 5 + i]; // Each character is 5 bytes wide
        i2c.write(SSD1306_I2C_ADDRESS, data, 2);
    }

    // Add a space between characters
    char space[2] = {0x40, 0x00};
    i2c.write(SSD1306_I2C_ADDRESS, space, 2);
}

void ssd1306_draw_string(int x, int y, const char *str) {
    while (*str) {
        ssd1306_draw_char(x, y, *str++);
    }
}

int main() {
    TemperatureModule TemperatureModule(D0, D1);
    AccelerationModule AccelerationModule(A6, A5, A4, D12);
    BuzzerModule BuzzerModulee(A2);
```

```
    TemperatureModule.configure();

   AccelerationModule.initialise();
        ssd1306_init();
    ssd1306_clear();

    while (true) {
        float temperature = TemperatureModule.readTemperature();
        float x, y, z;
        AccelerationModule.readAcceleration(x, y, z);
            ssd1306_init();
             ssd1306_clear();

        // 检测到异常情况
        if (temperature<20|temperature>29|x > 1.5 | y > 1.5 | z > 1.5 |
 x < -1.5 | y < -1.5 | z < -1.5) {
             ssd1306_init();
    ssd1306_clear();
     ssd1306_draw_string(0, 0, "Alert");
           break;
        }

        // ... 其他代码 ...
    }
    BuzzerModulee.alarm(frequency1, beat1, 28);
    return 0;
}
```

Testing Procedure

To evaluate the system's response to specific incidents, two distinct scenarios were tested: abnormal patient temperature and excessive acceleration along any of the xyz-axes. The objective was to determine if the buzzer and OLED would react appropriately to these incidents.

Temperature Monitoring Test: The TMP102 transducer was placed on a table to simulate normal conditions. The system was designed to trigger an alert if the patient's temperature deviated from the preset thresholds. To test this, the surrounding temperature of the transducer was incrementally increased by clutch it in the palm.

Acceleration Monitoring Test: The ADXL345 accelerometer was also placed on

a table to establish a baseline for normal conditions. The system was set to trigger an alert if acceleration along any axis exceeded the predefined values. To test this, the acceleration was dramatically increased by shaking it violently.

3.Results and Discussions

Testing Overview

The primary objective of the tests was to evaluate the system's response to two critical incidents: abnormal temperature readings and excessive acceleration values. The tests were designed to assess the functionality of the TemperatureModule using the TMP102 sensor and the AccelerationModule using the ADXL345 accelerometer, both integrated with a BuzzerModule and an OLED display for alert notifications.

Temperature Monitoring Results

The initial tests involved placing the TMP102 transducer on a table to simulate normal environmental conditions. The system was set to trigger an alert if the temperature deviated from the predefined set points. The results were as follows:

- **Normal Conditions Test:** At room temperature, the system did not trigger any alerts, indicating that the temperature thresholds were correctly configured and the system was functioning as expected under normal conditions.
- **Temperature Threshold Test:** Upon increasing the surrounding temperature, the TMP102 sensor accurately detected the change and triggered the alert mechanism. The buzzer started to ring, and the OLED display showed "Alert!", confirming a successful test for the temperature monitoring module.

Acceleration Monitoring Results

For the acceleration tests, the ADXL345 accelerometer was subjected to various conditions to evaluate its responsiveness to set incidents:

- **Static Test:** With the accelerometer at rest on a table, no alerts were triggered, which validated the system's baseline functionality in the absence of acceleration.
- **Dynamic Test:** Upon vigorously shaking the ADXL345 accelerometer, the acceleration in one or more axes exceeded the set threshold. The system

promptly responded by activating the buzzer and displaying "Alert!" on the OLED screen. This confirmed the successful operation of the acceleration monitoring module.

Discussion

the testing phase has validated the system's ability to detect and respond to critical incidents related to temperature and acceleration. The integration of the buzzer and OLED display for alert notifications ensures that any deviation from normal conditions is promptly communicated, providing a reliable solution for monitoring and alerting in various applications. This ensures the system meets the project's objectives of providing a robust, responsive monitoring solution.

4.Conclusions and Future Work

My project has successfully developed an energy-efficient monitoring system tailored for the elderly. Utilizing the NUCLEO L432KC platform, combined with the TMP102 for temperature readings and the ADXL345 for motion detection, I've created a reliable solution for real-time health surveillance. The integration of a buzzer and OLED display ensured timely alerts for deviations in body temperature or fall. However, there still remained some limitations for my project. The first part was that the codes about OLED were not modularized. The second limitation was that OLED could not demonstrate real-time temperature. In the future, I would try my best to solve these drawbacks and find some innovative and more energy-efficient solutions.

5.Reference

[1] V. Rogelj and D. Bogataj, "Ambient Assisted Living Technologies and Environments: Literature review and research agenda," in 2020 7th International Conference on Control, Decision and Information Technologies (CoDIT), Prague, Czech Republic, 2020, pp. 762-767, doi:

10.1109/CoDIT49905.2020.9263932.

[2] H. Chen and P. Li, "Intelligent Positioning Boots for Miners' Fall Monitoring Based on ZigBee Communication Protocol and ADXL345 Acceleration Sensor," 2022 IEEE Asia-Pacific Conference on Image Processing, Electronics and Computers (IPEC), Dalian, China, 2022, pp. 304-307, doi: 10.1109/IPEC54454.2022.9777577.